

Problema, motivação e objetivo

- **Problema:** as variações semanais das vendas dificultam o planejamento de abastecimento de lojas e departamentos da rede Walmart.
- **Motivação:** previsões mais precisas podem embasar decisões e melhorar o planejamento operacional.
- **Objetivo:** prever as vendas da próxima semana para cada loja e departamento do Walmart.
- **Contexto de utilização:** apoiar a priorização do abastecimento e a comparação entre demanda prevista e comportamento histórico.
- **Papel da IA:** aprender padrões históricos multivariados para gerar previsões de vendas semanais.

Equipe: Luis Eduardo Moreira Las Casas; Thiago Augusto Souza do Nascimento; Iuan Ferraz Souza; Natália Michelinini Monteiro; Gabriela Ladeira Nadaes; Gabriel Dantas Leite; Gabriel Lucena.

GitHub: <https://github.com/gabriela-ladeira-nadaes/pred-demanda-2026-1>

Dados e solução proposta

- **Base de dados:** [Walmart Sales Dataset](#)
- **Dimensão:** Originalmente com +400 mil amostras e 16 atributos
 - Cobertura: 45 lojas, 81 departamentos e 143 semanas
 - Granularidade: Uma observação por loja, departamento e semana
- **Variável alvo:** Weekly_Sales → Valor das vendas semanais de cada departamento em cada loja
- **Entrada:** informações da loja e do departamento, período do ano, ocorrência de feriado, promoções e condições econômicas.
- **Saída:** previsão das vendas da próxima semana para cada combinação de loja e departamento.
- **Solução proposta:**
 1. Limpar e ordenar cronologicamente os dados
 2. Criar atributos temporais a partir da data
 3. Codificar categorias e padronizar variáveis numéricas
 4. Separar treino e teste preservando a ordem temporal
 5. Treinar uma rede neural usando PyTorch.
 6. Otimizar o erro absoluto com peso maior para semanas de feriado.
 7. Gerar a previsão da próxima semana e apresentá-la por loja e departamento.

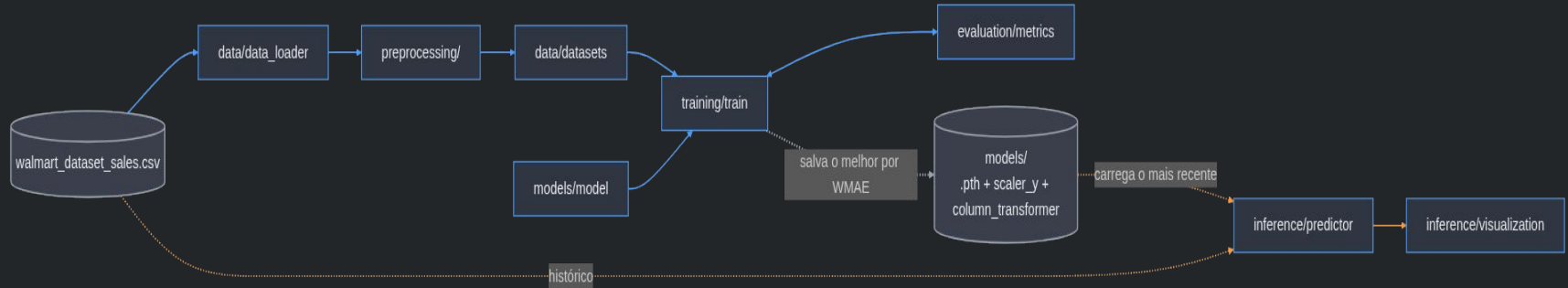
Requisitos Funcionais

#	Requisito Funcional	Insight/Decisão
RF01	O sistema deve gerar, semanalmente, a previsão de vendas da próxima semana para cada combinação de loja e departamento.	Planejamento semanal de abastecimento de lojas
RF02	O sistema deve permitir consultar as previsões de uma loja, de um departamento ou de toda a rede.	Análise direcionada
RF03	O sistema deve ordenar lojas e departamentos de acordo com as maiores e menores vendas previstas.	Priorização de abastecimento
RF04	O sistema deve registrar as vendas realizadas após cada período previsto e gerar uma comparação entre valores previstos e realizados.	Acompanhamento da qualidade das previsões
RF05	O sistema deve comparar cada previsão com as vendas realizadas na semana anterior e com o histórico da respectiva loja e departamento.	Identificação de variações

Requisitos não funcionais e critérios de aceitação

#	Requisito Não Funcional	Critério de Aceitação
RNF01	O sistema deve garantir alta qualidade preditiva, reduzindo o erro (WMAE) das previsões semanais em relação a um baseline.	No conjunto de teste, obter redução mínima de 15% no WMAE em relação ao baseline.
RNF02	O experimento deve ser reproduzível com a mesma versão dos dados, dependências, hiperparâmetros e semente aleatória.	Duas execuções com a mesma configuração devem apresentar uma diferença máxima de 5% no WMAE.
RNF03	O sistema deve produzir resultados completos e válidos para todas as combinações elegíveis de loja e departamento.	A avaliação deve confirmar 100% de cobertura, sem previsões ausentes ou inválidas.
RNF04	Os componentes de carregamento, preparação temporal, inferência, cálculo do WMAE e persistência devem poder ser testados de maneira independente.	Execução completa sem falhas e cada componente crítico pode ser testado isoladamente.
RNF05	O sistema deve ponderar as variações da demanda no contexto semanal em função de feriados.	Utilizar uma métrica de erro ponderada que dá maior peso a semanas com feriados.

Diagrama da arquitetura



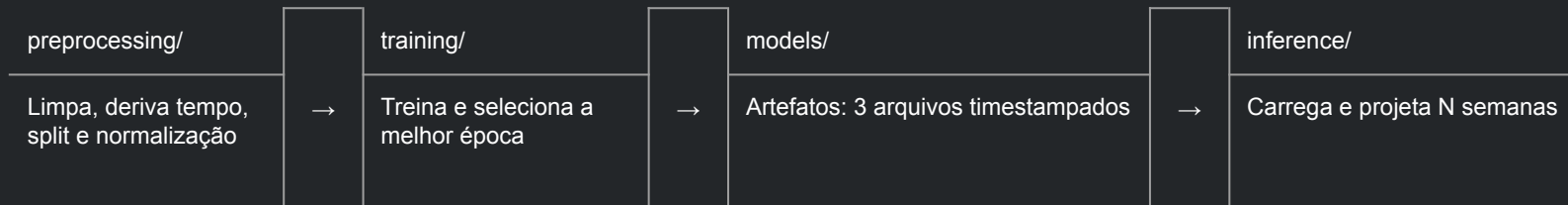
— Treino — Persistência — Inferência

Decisões arquiteturais

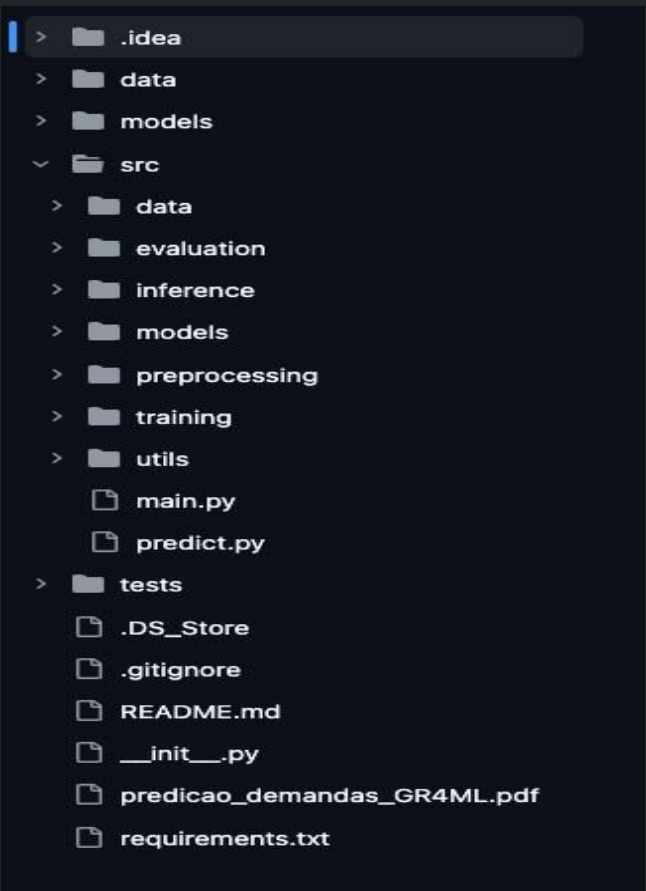
Cada pasta é uma etapa do ciclo de vida do dado, não uma camada técnica, e tem uma única razão para mudar.

SEPARAÇÃO DE RESPONSABILIDADES + ALTA COESÃO

Decisão	Evidência	Conceito
Módulos se comunicam por arrays e tensores, nunca por chamadas entre si	<code>preprocessing</code> não importa <code>training</code> , <code>data</code> não importa <code>models</code>	Baixo Acoplamento
Persistência em três arquivos	<code>.pth</code> + <code>scaler_y.pkl</code> + <code>column_transformer.pkl</code>	Reprodutibilidade
O pré-processador é ajustado só no treino e reusado na inferência	Testes unitários, ex: <code>test_standardize_fits_only_on_train</code>	Testabilidade
Caminhos, hiperparâmetros e listas de colunas num só lugar	<code>utils/config.py</code> , <code>BASE_DIR</code> a partir de <code>__file__</code>	Manutenabilidade
Treino e inferência são pontos de entrada distintos sobre os mesmos módulos	<code>main.py</code> treina · <code>predict.py</code> projeta	Reuso



Estrutura do projeto



Responsabilidades Claras

Cada módulo cuida de uma etapa: dados, preparação, modelos, treinamento, avaliação ou inferência.

Reutilização entre treino e previsão

Pesos e transformadores salvos podem ser carregados na previsão sem executar novamente o treinamento.

Manutenção e testes isolados

A separação facilita alterar e verificar componentes, sem concentrar toda a lógica em um único arquivo.

src/models/	models/
Código das arquiteturas	Pesos(.pth) e transformadores(.pkl)

src/main.py → coordena o treinamento | src/predict.py → executa a previsão

Responsabilidades dos Módulos

Módulo	Responsabilidade
data	Carrega e valida o CSV, converte os arrays NumPy em tensores, TensorDatasets e DataLoaders, seleciona CPU ou GPU.
preprocessing	Limpa e ordena os registros, cria Year, Month e WeekOfYear, realiza o split cronológico e transforma as variáveis.
models	Define os modelos LinearRegression, FinancialModel (MLP) e LSTMModel, salva e carrega pesos, scaler do alvo e transformador das features.
training	Executa o treinamento com Adam, loss L1 ponderada, gradient clipping e restauração da melhor época segundo o WMAE.
evaluation	Calcula o WMAE, atribuindo peso 5 aos erros em semanas de feriado e peso 1 às demais.
inference	Carrega o artefato mais recente, constrói os dados futuros, gera previsões semanais e produz gráficos históricos vs. projetados.
utils	Centraliza caminhos, semente, hiperparâmetros, data de corte, coluna-alvo e listas de features.
tests	Verifica carregamento, pré-processamento, ausência de vazamento, shapes dos tensores, saída dos modelos e persistência dos pesos.

Como main.py integra os componentes:

CSV > carregamento e validação > limpeza e features temporais > split cronológico e normalização > tensores e DataLoaders > modelo > treinamento e WMAE > persistência dos artefatos

Código Python, NumPy e tipagem

Construção da matriz de features:

```
def build_features_matrix(  
    data: pd.DataFrame  
) -> tuple[np.ndarray, np.ndarray]:  
    """Monta arrays NumPy para a matriz de features (X)  
    e o vetor alvo (y)."""  
    feature_cols = (  
        NUMERIC_FEATURES  
        + CATEGORICAL_FEATURES  
        + TEMPORAL_FEATURES  
    )  
    X = data[feature_cols].to_numpy(dtype = np.float32)  
    y = data[TARGET_COLUMN].to_numpy(dtype = np.float32)  
    return X, y
```

Métrica de erro:

```
def wmae_score_numpy(  
    y_true: np.ndarray,  
    y_pred: np.ndarray,  
    is_holiday: np.ndarray  
) -> float:  
    """Calcula o Erro Absoluto Medio Ponderado (WMAE)  
    usando NumPy"""  
    pesos = np.where(is_holiday == 1, 5, 1)  
    erro_absoluto = np.abs(y_true - y_pred)  
    return np.sum(pesos * erro_absoluto) / np.sum(pesos)
```

Separação cronológica de conjuntos de treino e teste:

```
def split_data(  
    data: pd.DataFrame,  
    cutoffDate: datetime = CUTOFF_DATE  
) -> tuple[  
    np.ndarray, np.ndarray, np.ndarray, np.ndarray  
]:  
    """Divide dados de treino e teste ordenados  
    cronologicamente para evitar vazamento"""  
    df_train = data[data['Date'] < cutoffDate].copy()  
    df_test = data[data['Date'] >= cutoffDate].copy()  
    return (  
        build_features_matrix(df_train) + build_features_matrix(df_test)  
    )
```

Implementação do Modelo de IA em PyTorch

```
class FinancialModel(nn.Module):
    def __init__(self, input_dim: int, output_dim: int):
        super(FinancialModel, self).__init__()

        self.net = nn.Sequential(
            # Primeira camada oculta
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),
            nn.Dropout(0.4),

            # Segunda camada oculta
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.BatchNorm1d(64),
            nn.Dropout(0.4),

            # Terceira camada oculta
            nn.Linear(64, 32),
            nn.ReLU(),

            # Camada de Saída (1 neurônio para prever Weekly_Sales)
            nn.Linear(32, output_dim)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

● ARQUITETURA

MLP totalmente conectado: $200 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1$, com função de ativação ReLU em todas as camadas

● FUNÇÃO DE PERDA

L1 ponderada, alinhada à métrica de negócio (WMAE) - peso 5x em semanas de feriado

● OTIMIZADOR

ADAM | Learning Rate: 0,00015 | Weight Decay: 0,0001

● ÉPOCAS

50 por modelo candidato

● BATCH SIZE

64 amostras por lote (DataLoader)

● CPU / GPU

Seleção automática - usa CUDA se disponível, senão CPU

● SALVAMENTO E CARREGAMENTO

Salvamento implementado em .pth, versionado por timestamp.

Configuração Experimental

294.132

registros de treino

fev/2010 – dez/2011

127.438

registros de teste

jan/2012 – out/2012

100

épocas por modelo

3 modelos candidatos comparados

Hiperparâmetros e Métricas

- **BATCH SIZE:** 64
- **OTIMIZADOR:** Adam ($\text{lr} = 0,0001$, $\text{weight_decay} = 0,0001$)
- **DROPOUT:** 0,4 nas duas primeiras camadas ocultas
- **GRADIENT CLIPPING:** $\text{max_norm} = 5,0$
- **MÉTRICAS DE AVALIAÇÃO:** RMSE, MAE, WMAE (peso 5x em feriado) e R^2

Configurações Comparadas

- **FUNÇÃO DE PERDA**

MSE e L1 ponderada (alinhada à WMAE)

- **GRADIENT CLIPPING**

Sem clipping e com clipping $\text{max_norm} = 5,0$ (ajustado a partir de 1,0)

- **MODELO**

Regressão Linear (baseline), LSTM e MLP

O QUE BUSCÁVAMOS DESCOBRIR

Se alinhar a loss de treino à métrica de avaliação de negócio (WMAE) e limitar a norma do gradiente reduziria a oscilação do erro entre épocas — sem comprometer a velocidade de convergência do modelo.

Exemplos de testes e evidências

```
class TestPreprocessing(unittest.TestCase):
    def setUp(self):
        self.raw = make_sample_dataframe()

    def _prepared(self) -> pd.DataFrame:
        return create_features(clean_data(self.raw))

    def test_clean_data_drops_unnamed_and_sorts(self):
        shuffled = self.raw.sample(frac=1.0, random_state=1).reset_index(drop=True)
        out = clean_data(shuffled)

        self.assertNotIn("Unnamed: 0", out.columns)
        self.assertTrue(pd.api.types.is_datetime64_any_dtype(out[DATE_COLUMN]))

        # Ordenado por Store, Dept e Date e reindexado de 0..n-1.
        expected = out.sort_values(SORT_COLUMNS).reset_index(drop=True)
        pd.testing.assert_frame_equal(out, expected)
        self.assertTrue((out.index == np.arange(len(out))).all())
```

```
def test_create_features_adds_temporal(self):
    cleaned = clean_data(self.raw)
    out = create_features(cleaned)

    for col in TEMPORAL_FEATURES:
        self.assertIn(col, out.columns)

    first_date = cleaned[DATE_COLUMN].iloc[0]
    self.assertEqual(int(out["Year"].iloc[0]), first_date.year)
    self.assertEqual(int(out["Month"].iloc[0]), first_date.month)
    self.assertEqual(int(out["WeekOfYear"].iloc[0]), first_date.isocalendar().week)
```

Exemplos selecionados

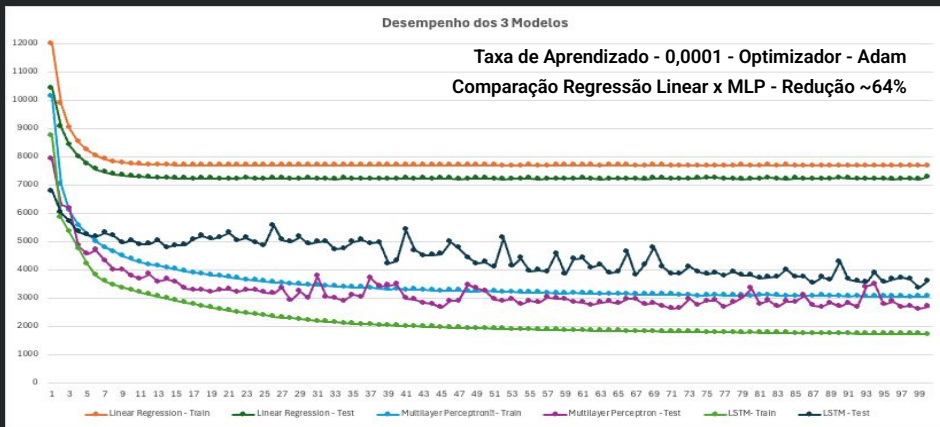
- Carregamento dos dados
test_load_data_reads_csv
test_validate_data_empty
→ test_data_loader.py
- Pré-processamento
test_standardize_shape
test_standardize_mean
→ test_preprocessing.py
- Regra específica do sistema
test_split_is_chronological
test_standardize_fits_only_on_train
→ test_system_rules.py

Evidência da execução

Ran 25 tests in 1.082s

OK

Resultados, conclusão e limitações



- Requisitos Funcionais e não funcionais atingidos. Ex. RF01 e RN01
- Limitação do MLP que responde bem globalmente, mas não é bom para entender impactos temporais. Ex. Pico de venda em uma semana
- Próximos Passos:
 - Testar novos modelos como baseado em árvores - Ex.: XGBoost e Random Forest
 - Criar novas colunas no pré-processamento. Ex: volume de semanas anteriores
 - Automatizar o re-treino para com o fechamento mensal

Modelo	Época	RMSE	MAE	WMAE	R2
Linear Regression	Train	14521,13	7411,99	7680,21	0,6
	Test	13049,88	7126,73	7193,12	0,652
MLP	Train	6187,18	2895,19	3050,73	0,9274
	Test	5183,67	2534,74	2604,71	0,9451
LSTM	Train	4009,26	1647,18	1715,51	0,9695
	Test	5846,49	3199,35	3351,25	0,9301

